Project Specification: react-native-turbo-sse

This document serves as the architectural blueprint and implementation guide for building a high-performance, native-backed Server-Sent Events (SSE) client for React Native using the New Architecture (TurboModules/JSI). This specification contains everything an AI agent or engineer needs to scaffold and build the package.

1. Executive Summary & Problem Statement

Current market solutions for SSE in React Native (such as react-native-sse) are written entirely in JavaScript. They polyfill the EventSource API by wrapping React Native's built-in XMLHttpRequest (XHR).

Because standard XHR requests pass through the asynchronous React Native network bridge, the native OS layer aggressively buffers incoming data streams. Instead of delivering a smooth, token-by-token streaming effect required for modern Large Language Model (LLM) interfaces, text arrives with significant delays and in large, unpredictable blocks. Furthermore, these connections frequently drop when the application enters a background state.

The Solution: react-native-turbo-sse delegates connection management completely to low-level native networking frameworks, pumping data fragments synchronously into the JavaScript runtime via C++ JSI hooks, guaranteeing 0-buffer, sub-millisecond event dispatching.

2. Architectural Comparison

The difference in data flow between the traditional architecture and this high-performance JSI implementation is detailed below:

Feature	Traditional JS-Polyfill (XHR)	TurboModule + JSI (This Package)
Networking Layer	JS-wrapped XHR Polyfill	Native OS Core (OkHttp / NSURLSession)
Data Transfer Method	Asynchronous JSON Bridge Serialization	Synchronous Memory Invocations via JSI (C++)

Feature	Traditional JS-Polyfill (XHR)	TurboModule + JSI (This Package)
Buffering Behavior	High (Aggregated bulk chunks)	**Zero Buffering (Instantaneous token streaming)**
Background Performance	Instantly killed when JS thread pauses	Maintained by OS-level connection lifecycle

3. Technical Implementation Specification

A. Android Native Layer (Kotlin & OkHttp)

Use OkHttp's official okhttp-sse extension library. Avoid manual string splitting of HTTP chunks.

```
dependencies {
    implementation "com.squareup.okhttp3:okhttp:4.12.0"
    implementation "com.squareup.okhttp3:okhttp-sse:4.12.0"
}
```

Implement the EventSourceListener interface to handle lifecycle events natively:

```
override fun onEvent(eventSource: EventSource, id: String?, type:
String?, data: String) {
    // Synchronously forward data over C++ JSI context directly to
JS
    jsiEmitEvent(type ?: "message", id, data)
}
```

B. iOS Native Layer (Swift / Objective-C & NSURLSession)

Utilize NSURLSessionDataDelegate. To explicitly prevent network chunk aggregation, configure the stream configuration delegate to process data segments textually as soon as byte arrays fill up.

```
func urlSession(_ session: URLSession, dataTask:
URLSessionDataTask, didReceive data: Data) {
    if let rawString = String(data: data, encoding: .utf8) {
        // Parse the standard "id:", "event:", "data:" protocol
```

```
markers
        // Instantly call the JSI runtime function hook
    }
}
```

4. JavaScript / TypeScript API Architecture

The public API surface must emulate standard browser specifications while adding strict TypeScript typing for AI token payloads.

```
export interface TurboSSEOptions {
  method?: 'GET' | 'POST';
  headers?: Record<string, string>;
  body?: string;
}

export class TurboEventSource {
  constructor(url: string, options?: TurboSSEOptions);

  onOpen(callback: () => void): void;
  onMessage(callback: (event: { id: string; data: string }) =>
void): void;
  onError(callback: (error: Error) => void): void;

  close(): void;
}
```

5. Guide for the AI Developer Agent

When generating code blocks for this module, execute the following implementation order:

1. **Codegen Schema Definition:** Define the NativeTurboSSE spec file using standard Flow/TypeScript definitions for the New Architecture.
2. **C++ JSI Registration:** Create the host object binding so the native side can run JavaScript functions inside callback triggers directly without going through the RCTDeviceEventEmitter system.
3. **Payload Forwarding:** Ensure the native string streams trim carriage returns (\r\n) accurately to conform precisely with the server-sent text specification protocols before piping them into the runtime.